

Auteur : GCI – Département Formation & Intelligence Analytique  
Version : 1.0 – Édition Complète  
Format : Bible Professionnelle  
Langue : Français

=====

## TABLE DES MATIÈRES

|             |                                                             |
|-------------|-------------------------------------------------------------|
| PARTIE I    | – LA DATA SCIENCE AU CŒUR DE L'IA : CARTOGRAPHIE DES ENJEUX |
| PARTIE II   | – COMPRENDRE CE QU'EST UN DATASET                           |
| PARTIE III  | – CRÉER UN DATASET DE ZÉRO : MÉTHODES ET PRATIQUES          |
| PARTIE IV   | – COLLECTER LA DONNÉE : SOURCES ET TECHNIQUES               |
| PARTIE V    | – NETTOYER ET PRÉPARER UN DATASET                           |
| PARTIE VI   | – ANNOTER ET ÉTIQUETER LES DONNÉES                          |
| PARTIE VII  | – ÉQUILIBRE, BIAIS ET REPRÉSENTATIVITÉ                      |
| PARTIE VIII | – FORMATS, STOCKAGE ET VERSIONNEMENT                        |
| PARTIE IX   | – DATASETS POUR CHAQUE TYPE D'IA                            |
| PARTIE X    | – ENJEUX ÉTHIQUES ET LÉGAUX                                 |
| PARTIE XI   | – PIPELINES DE DONNÉES EN PRODUCTION                        |
| PARTIE XII  | – DATASETS DANS LE CONTEXTE AFRICAIN ET CONGOLAIS           |
| PARTIE XIII | – CHECKLIST DU DATA SCIENTIST : DU DATASET AU MODÈLE        |

## PARTIE I – LA DATA SCIENCE AU CŒUR DE L'IA : CARTOGRAPHIE DES ENJEUX

=====

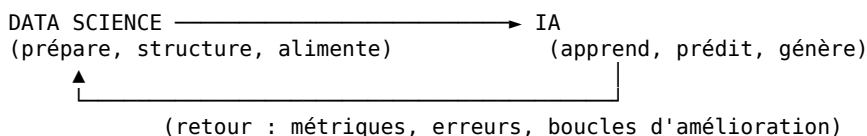
### 1.1 LE RAPPORT FONDAMENTAL ENTRE DATA SCIENCE ET IA

=====

L'intelligence artificielle moderne ne pense pas. Elle apprend.  
Et pour apprendre, elle a besoin d'exemples – des données.

La data science est la discipline qui prépare, structure, analyse et fournit cette matière première. Sans data science, l'IA est un moteur sans carburant. Sans IA, la data science reste descriptive.

Leur relation est symbiotique :



Le data scientist qui travaille pour l'IA a un rôle particulier :  
il ne cherche pas seulement à comprendre la donnée, il cherche à la rendre APPRENANTE – c'est-à-dire utilisable par un algorithme pour produire des comportements intelligents.

### 1.2 LES TROIS ÈRES DE L'INTELLIGENCE ARTIFICIELLE

=====

Pour comprendre les enjeux actuels, il faut situer l'histoire.

#### ÈRE 1 – L'IA SYMBOLIQUE (1950–1990)

=====

L'IA était construite avec des règles écrites à la main par des experts.  
Exemple : "SI température > 38°C ET toux = vrai ALORS diagnostic = grippe"

- Pas besoin de données massives.
- Limité : les règles ne couvrent jamais tous les cas réels.

#### ÈRE 2 – L'IA STATISTIQUE ET LE MACHINE LEARNING (1990–2012)

=====

Les algorithmes apprennent des patterns à partir de données.  
Exemple : régression, arbres de décision, SVM.

- Les données deviennent nécessaires mais en quantité modérée.

→ La qualité prime sur la quantité.

### ÈRE 3 – L'IA PAR APPRENTISSAGE PROFOND (2012–aujourd'hui)

Les réseaux de neurones profonds apprennent des représentations complexes.  
Exemples : GPT, DALL-E, AlphaFold, Stable Diffusion.

- Besoin de données massives, diverses, bien préparées.
- La DATA est devenue le vrai avantage compétitif.

#### LEÇON CENTRALE :

Plus l'IA devient puissante, plus la qualité du dataset devient critique.  
Un mauvais dataset produit une IA inutile – même avec le meilleur algorithme.

---

### 1.3 LES GRANDS ENJEUX DE LA DATA SCIENCE DANS L'IA

---

#### ENJEU 1 – LA QUALITÉ AVANT LA QUANTITÉ

"Garbage in, garbage out" est la loi fondamentale de l'IA.  
Un modèle entraîné sur des données corrompues, biaisées ou mal étiquetées produira des résultats erronés avec une grande confiance.  
C'est plus dangereux qu'un modèle qui dit "je ne sais pas".

#### ENJEU 2 – LA REPRÉSENTATIVITÉ

Un dataset qui ne reflète pas la réalité du terrain fausse l'IA.  
Exemple concret : un système de reconnaissance faciale entraîné uniquement sur des visages blancs américains échouera sur des visages africains. Ce n'est pas un problème d'algorithme – c'est un problème de dataset.

#### ENJEU 3 – LE BIAIS SYSTÉMIQUE

Les données reflètent les inégalités du monde réel.  
Si historiquement les femmes ont été moins promues, un modèle RH entraîné sur ces données va reproduire et amplifier cette inégalité.  
Le data scientist a une responsabilité éthique active.

#### ENJEU 4 – LA RARETÉ DES DONNÉES LABELLISÉES

Annoter (étiqueter) des données coûte cher et prend du temps.  
80% du travail d'un projet d'IA est dans la préparation des données.  
Seulement 20% est dans la modélisation elle-même.

#### ENJEU 5 – LA CONFIDENTIALITÉ ET LA SOUVERAINETÉ

Les données médicales, financières, personnelles sont réglementées.  
Les pays et entreprises qui contrôlent leurs données contrôlent leur avenir.  
C'est un enjeu géopolitique, pas seulement technique.

#### ENJEU 6 – LA DÉRIVE DES DONNÉES (DATA DRIFT)

Le monde change. Une IA entraînée en 2020 peut être obsolète en 2024 si les données de départ ne reflètent plus la réalité actuelle.  
Les pipelines de données doivent être maintenus vivants.

#### ENJEU 7 – L'EXPLICABILITÉ

Quand une IA prend une décision (crédit refusé, diagnostic, condamnation), qui peut expliquer pourquoi ? La donnée utilisée est souvent opaque.  
Les régulateurs exigent de plus en plus de transparence.

---

### 1.4 TAXONOMIE DE L'IA : QUI A BESOIN DE QUELS TYPES DE DONNÉES ?

---

| TYPE D'IA                 | DONNÉES NÉCESSAIRES                                   |
|---------------------------|-------------------------------------------------------|
| Vision par ordinateur     | Images annotées (bounding boxes, masques, labels)     |
| NLP / LLM                 | Textes massifs, paires question-réponse, instructions |
| Audio / Parole            | Fichiers audio + transcriptions                       |
| Recommandation            | Historiques de comportements, ratings, interactions   |
| Prévision (forecasting)   | Séries temporelles (dates, valeurs, événements)       |
| Détection d'anomalies     | Données "normales" + quelques cas anormaux            |
| Reinforcement Learning    | Environnements simulés, récompenses définies          |
| Génération (images/texte) | Paires (prompt → output), données créatives annotées  |

---

## PARTIE II – COMPRENDRE CE QU'EST UN DATASET

---

---

## 2.1 DÉFINITION RIGoureuse

---

Un dataset (jeu de données) est une collection organisée d'observations, chacune décrivant une entité du monde réel selon un ensemble de variables.

### STRUCTURE DE BASE :

- Une ligne (row / sample / observation / exemple) = une entité
- Une colonne (feature / variable / attribut) = une caractéristique
- Une cellule = la valeur d'une variable pour une entité donnée

### Exemple minimal :

| ID  | Âge | Revenu_FCFA | A_remboursé |
|-----|-----|-------------|-------------|
| 001 | 34  | 85 000      | Oui         |
| 002 | 27  | 42 000      | Non         |
| 003 | 51  | 150 000     | Oui         |

### Dans ce dataset :

- 3 observations (clients)
- 3 features (âge, revenu, remboursement)
- La colonne "A\_remboursé" est le LABEL (ce qu'on veut prédire)
- Les autres colonnes sont les FEATURES (ce qu'on utilise pour prédire)

---

## 2.2 LES TYPES DE DATASETS

---

### [A] PAR STRUCTURE

|            |                                                          |
|------------|----------------------------------------------------------|
| Tabulaire  | → Tableau classique lignes × colonnes (CSV, Excel, SQL)  |
| Temporel   | → Données indexées dans le temps (séries chronologiques) |
| Textuel    | → Documents, articles, conversations, tweets             |
| Image      | → Fichiers JPEG/PNG organisés avec métadonnées           |
| Audio      | → Fichiers WAV/MP3 avec transcriptions ou labels         |
| Vidéo      | → Séquences d'images avec annotations                    |
| Graphe     | → Nœuds et arêtes (réseaux sociaux, molécules)           |
| Géospatial | → Coordonnées GPS, polygones, cartes                     |

### [B] PAR MODE D'APPRENTISSAGE

|                  |                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Supervisé        | → Chaque exemple a un label connu<br>(email → spam/non-spam, image → chien/chat)                                    |
| Non supervisé    | → Pas de labels, l'IA découvre des structures<br>(clustering de clients, détection de thèmes)                       |
| Semi-supervisé   | → Peu de données labellisées + beaucoup de non-labellisées<br>(cas fréquent dans la vraie vie : annoter coûte cher) |
| Auto-supervisé   | → Le label est généré depuis les données elles-mêmes<br>(GPT : prédire le mot suivant à partir du texte)            |
| Par renforcement | → Séquences d'états, actions, récompenses<br>(jeu, robotique, optimisation)                                         |

### [C] PAR TAILLE

|               |                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------|
| Petit dataset | → < 10 000 exemples<br>→ Techniques classiques (régression, RF, SVM)<br>→ Attention au surapprentissage     |
| Moyen dataset | → 10 000 – 1 million d'exemples<br>→ ML et deep learning léger                                              |
| Grand dataset | → > 1 million d'exemples<br>→ Deep learning, infrastructure distribuée<br>→ Coût de traitement significatif |
| Mega dataset  | → Milliards d'exemples (Common Crawl, ImageNet élargi)<br>→ Réservé aux LLM, nécessite TPU/GPU clusters     |

---

## 2.3 ANATOMIE D'UN DATASET POUR L'IA

---

Un dataset complet et professionnel contient :

- [1] LES DONNÉES ELLES-MÊMES  
Fichiers CSV, images, audio, textes, JSON, Parquet, etc.
- [2] UN FICHER README  
Description du dataset, origine, date de création, licence, liste des variables, unités, valeurs possibles, limitations connues.

- [3] UN DICTIONNAIRE DES DONNÉES (Data Dictionary)  
Tableau expliquant chaque variable :

| Variable       | Type    | Description             | Exemple |
|----------------|---------|-------------------------|---------|
| age            | entier  | Âge du client en années | 34      |
| revenu_mensuel | réel    | Revenu en FCFA          | 85000.0 |
| default_credit | binaire | 1=défaut, 0=remboursé   | 0       |

- [4] LES SPLITS (PARTITIONS)
  - Train set → Données d'entraînement (70-80%)
  - Validation set → Réglage des hyperparamètres (10-15%)
  - Test set → Évaluation finale (10-20%)

▲ RÈGLE ABSOLUE : le test set ne doit JAMAIS être vu pendant l'entraînement.  
C'est le juge impartial. Le contaminer invalide tous les résultats.

- [5] LES MÉTADONNÉES DE QUALITÉ
  - Taux de valeurs manquantes par colonne
  - Distribution des classes (équilibré / déséquilibré ?)
  - Date de collecte
  - Source et méthode de collecte
  - Transformations déjà appliquées

---

## PARTIE III – CRÉER UN DATASET DE ZÉRO : MÉTHODES ET PRATIQUES

---

---

### 3.1 LE PROCESSUS EN 7 ÉTAPES

---

Créer un dataset n'est pas une tâche technique. C'est un projet complet qui demande autant de rigueur qu'un projet de développement logiciel.

#### ÉTAPE 1 – DÉFINIR L'OBJECTIF

Avant toute collecte : Quel problème veut-on résoudre ?  
→ "Prédire si un client va rembourser son crédit"  
→ "Classifier les maladies des cultures de manioc au Congo"  
→ "Détecter les fraudes dans les paiements mobile money"

L'objectif définit :

- Quelles variables collecter
- Quel type de label utiliser
- Quelle taille de dataset viser
- Quelles populations échantillonner

PIÈGE CLASSIQUE : Collecter des données "au cas où" sans objectif clair.  
Résultat : des téraoctets de données inutilisables.

#### ÉTAPE 2 – IDENTIFIER LES SOURCES

Voir Partie IV pour le détail complet.  
En résumé : bases de données internes, scraping web, APIs, formulaires, capteurs IoT, datasets publics, crowdsourcing.

#### ÉTAPE 3 – DÉFINIR LE SCHÉMA

Avant de collecter, décider exactement :

- Quelles colonnes (variables) ?
- Quel type de données pour chacune ?

- Quelles valeurs sont acceptables ?
- Comment traiter les valeurs manquantes ou aberrantes ?

Ce schéma est comme un contrat. Ne pas le changer en cours de route sauf si une révision formelle est justifiée.

#### ÉTAPE 4 – COLLECTER

Méthode, outils, scripts, automatisation.  
Voir Partie IV.

#### ÉTAPE 5 – NETTOYER ET PRÉPARER

Suppression des doublons, correction des erreurs, imputation des manquants.  
Voir Partie V.

#### ÉTAPE 6 – ANNOTER / ÉTIQUETER

Si apprentissage supervisé : attribuer les labels.  
Voir Partie VI.

#### ÉTAPE 7 – VALIDER ET DOCUMENTER

Vérifier la cohérence, tester sur un modèle simple,  
rédiger le README et le dictionnaire des données.

---

### 3.2 DÉFINIR LE BON OBJECTIF : LA QUESTION AVANT LES DONNÉES

---

La tentation est de commencer par "quelles données j'ai ?"  
La bonne pratique est de commencer par "quel problème je résous ?"

FRAMEWORK DE DÉFINITION D'OBJECTIF :

- [1] PROBLÈME MÉTIER  
"Notre taux de défaut de crédit est trop élevé"
- [2] TRADUCTION EN TÂCHE ML  
"Classifier les clients en : risque élevé / risque faible"
- [3] DÉFINITION DU LABEL  
Que signifie concrètement "défaut" ?  
→ Retard > 30 jours ? > 90 jours ? Non-remboursement total ?  
Réponse précise obligatoire avant toute collecte.
- [4] DÉFINITION DES FEATURES CANDIDATES  
Âge, revenu, historique de paiement, type d'emploi, localisation...  
Lesquelles sont disponibles ? Lesquelles sont légales à utiliser ?
- [5] DÉFINITION DU SUCCÈS  
"Un modèle qui prédit correctement 80% des défauts  
sans faussement rejeter plus de 10% des bons clients"

Sans cette étape, le dataset sera incomplet ou inadapté.

---

### 3.3 TAILLE MINIMALE : COMBIEN DE DONNÉES FAUT-IL ?

---

Il n'existe pas de règle universelle. Voici des ordres de grandeur pratiques.

#### CLASSIFICATION BINAIRE (2 classes)

|                                              |                             |
|----------------------------------------------|-----------------------------|
| Modèle simple (régression logistique, arbre) | → 500 – 2 000 exemples      |
| Modèle ML classique (Random Forest, SVM)     | → 2 000 – 10 000 exemples   |
| Réseau de neurones léger                     | → 10 000 – 100 000 exemples |
| Deep learning avancé                         | → 100 000+ exemples         |

#### CLASSIFICATION MULTI-CLASSE (N classes)

Règle de base : 1 000 exemples par classe minimum pour ML classique.  
Exemple : 10 catégories de produits → 10 000 exemples minimum.

RÉGRESSION (prédire une valeur continue)

---

10× le nombre de features minimum.

Exemple : 20 variables → au moins 200 exemples (mais viser 2 000+).

## DÉTECTION D'ANOMALIES

---

Les anomalies sont rares par définition.

Viser 5 à 10% d'anomalies dans le dataset (sur-échantillonner si nécessaire).

## VISION PAR ORDINATEUR

---

Transfer learning (partir d'un modèle pré-entraîné) : 200–500 images/classe

Entraînement complet : 10 000–100 000 images/classe

## NLP / TEXTE

---

Tâches simples (sentiment) avec fine-tuning : 1 000–5 000 exemples

LLM from scratch : milliards de tokens (hors de portée individuelle)

---

## PARTIE IV – COLLECTER LA DONNÉE : SOURCES ET TECHNIQUES

---

---

### 4.1 LES GRANDES SOURCES DE DONNÉES

---

#### [A] DONNÉES INTERNES (la source la plus précieuse)

Ce sont les données générées par l'organisation elle-même :

- Logs des systèmes (transactions, connexions, erreurs)
- CRM (historique clients, achats, contacts)
- ERP (stocks, fournisseurs, comptabilité)
- Applications propriétaires (formulaires, interfaces)

#### AVANTAGES :

- Données réelles du contexte métier exact
- Pas de problème de licence
- Continuellement actualisées

#### DÉFIS :

- Souvent non structurées ou mal documentées
- Données cloisonnées dans différents systèmes
- Nécessitent un accès et des autorisations

#### [B] DONNÉES PUBLIQUES ET OPEN DATA

Datasets librement accessibles :

- Kaggle ([kaggle.com/datasets](https://kaggle.com/datasets))
- UCI Machine Learning Repository
- Google Dataset Search
- Hugging Face Datasets
- Data.gouv.fr, Eurostat
- INS Congo, Banque Mondiale (Afrique)
- OpenStreetMap (données géographiques)
- CommonCrawl, Wikipedia dumps (NLP)

AVANTAGES : Gratuites, variées, souvent documentées.

LIMITES : Génériques, pas adaptées au contexte local sans adaptation.

#### [C] WEB SCRAPING (extraction automatisée du web)

##### PRINCIPE :

Écrire un programme qui visite des pages web et en extrait des informations structurées.

##### OUTILS PYTHON :

- BeautifulSoup : parsing HTML simple
- Scrapy : framework scraping complet
- Selenium : sites dynamiques (JavaScript)
- Playwright : alternative moderne à Selenium

##### EXEMPLE SIMPLE AVEC BEAUTIFULSOUP :

```
import requests
```

```

from bs4 import BeautifulSoup
import pandas as pd

url = "https://exemple-marche.cg/produits"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

produits = []
for item in soup.find_all("div", class_="produit"):
    nom = item.find("h2").text.strip()
    prix = item.find("span", class_="prix").text.strip()
    produits.append({"nom": nom, "prix": prix})

df = pd.DataFrame(produits)
df.to_csv("produits_marche.csv", index=False)

```

#### RÈGLES ÉTHIQUES ET LÉGALES DU SCRAPING :

- Vérifier le fichier robots.txt du site
- Ne pas surcharger le serveur (délais entre requêtes)
- Respecter les CGU (certains sites l'interdisent)
- Ne pas scraper de données personnelles sans consentement

#### [D] APIs (Application Programming Interface)

Une API est une interface qui permet d'interroger une source de données de manière structurée et autorisée.

#### EXEMPLES D'APIS UTILES :

- Twitter/X API : tweets, tendances, profils
- Google Maps API : lieux, distances, itinéraires
- OpenWeatherMap : données météo historiques et temps réel
- Alpha Vantage : données boursières
- Twilio : données SMS (avec accord)

#### EXEMPLE D'APPEL API :

```

import requests
import pandas as pd

API_KEY = "votre_clé_api"
ville = "Brazzaville"
url = f"https://api.openweathermap.org/data/2.5/weather"
params = {"q": ville, "appid": API_KEY, "units": "metric"}

response = requests.get(url, params=params)
data = response.json()
print(f"Température : {data['main']['temp']}°C")

```

#### [E] COLLECTE MANUELLE ET FORMULAIRES

Pour des données locales sans source digitale préexistante :

- Google Forms / Kobo Toolbox (enquêtes terrain)
- ODK (Open Data Kit) pour les enquêtes mobiles en zones rurales
- Fiches papier + saisie manuelle

#### APPLICABLE AU CONTEXTE CONGOLAIS :

- Enquêtes auprès des vendeurs de marché
- Relevés de prix mensuels
- Données médicales dans des centres de santé non connectés

#### [F] CAPTEURS ET IoT

Appareils physiques qui génèrent des données en continu :

- Capteurs de température/humidité (agriculture)
- Compteurs électriques intelligents
- GPS (véhicules, livraisons)
- Caméras de surveillance

#### [G] DONNÉES SYNTHÉTIQUES

Données générées artificiellement pour compléter un dataset insuffisant.  
Voir section 4.3.

---

## 4.2 STRATÉGIES DE COLLECTE SELON LE CONTEXTE

---

#### CONTEXTE : FAIBLES DONNÉES DISPONIBLES

- Commencer par les données internes existantes
- Enrichir avec des sources publiques
- Générer des données synthétiques pour compléter
- Utiliser le transfer learning pour contourner le besoin de grosses données

#### CONTEXTE : DONNÉES SENSIBLES (médical, financier)

- Anonymisation avant tout traitement
- Pseudonymisation des identifiants
- Vérifier la réglementation locale et internationale
- Données synthétiques comme alternative

#### CONTEXTE : DONNÉES TEXTUELLES LOCALES (langues africaines)

- Wikidata / Wikipedia en lingala, kikongo, teke : ressources limitées
- Collecte communautaire (crowdsourcing)
- Partenariats avec universités et médias locaux
- Radio Congo, Adiac-Congo : archives textuelles potentielles

---

### 4.3 DONNÉES SYNTHÉTIQUES : GÉNÉRER CE QU'ON N'A PAS

---

#### DÉFINITION :

Les données synthétiques sont des données artificiellement générées qui imitent les propriétés statistiques de données réelles.

#### CAS D'USAGE :

- Manque de données réelles (contexte africain, domaines spécialisés)
- Protection de la vie privée (simuler des données médicales)
- Tester un pipeline avant d'avoir les vraies données
- Équilibrer un dataset déséquilibré (SMOTE)

#### MÉTHODES :

##### [1] BIBLIOTHÈQUE FAKER (données tabulaires)

```
from faker import Faker
import pandas as pd
import random

fake = Faker("fr_FR")

clients = []
for _ in range(1000):
    clients.append({
        "nom": fake.last_name(),
        "prenom": fake.first_name(),
        "age": random.randint(18, 65),
        "ville": random.choice(["Brazzaville", "Pointe-Noire",
                                "Dolisie", "Nkayi"]),
        "revenu_fcfa": random.randint(30000, 500000),
        "default": random.choices([0, 1], weights=[0.8, 0.2])[0]
    })

df = pd.DataFrame(clients)
df.to_csv("clients_synthetiques.csv", index=False)
```

##### [2] SMOTE (Synthetic Minority Over-sampling Technique) Pour équilibrer des classes déséquilibrées.

```
from imblearn.over_sampling import SMOTE

X = df.drop("label", axis=1)
y = df["label"]

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
```

##### [3] GANs (Generative Adversarial Networks)

Pour données complexes : images, textes, données tabulaires avancées. Deux réseaux s'affrontent : un générateur crée des données fausses, un discriminateur tente de les détecter.  
→ Résultat : données synthétiques réalistes.



- [4] SDV (Synthetic Data Vault) – bibliothèque Python  
Génère des données tabulaires synthétiques avec relations  
entre tables préservées.

```
from sdv.single_table import GaussianCopulaSynthesizer

synthesizer = GaussianCopulaSynthesizer(metadata)
synthesizer.fit(real_data)
synthetic_data = synthesizer.sample(num_rows=1000)
```

---

## PARTIE V – NETTOYER ET PRÉPARER UN DATASET

---

---

### 5.1 POURQUOI LE NETTOYAGE EST CRITIQUE

---

Les études de terrain montrent que les data scientists passent  
60 à 80% de leur temps à nettoyer et préparer les données.

Ce n'est pas un travail ingrat. C'est LE travail fondamental.  
Un modèle sur des données propres bat toujours un modèle sophistiqué  
sur des données sales.

---

### 5.2 INVENTAIRE DES PROBLÈMES DE QUALITÉ

---

- [1] VALEURS MANQUANTES (NULL, NaN, vide)  
Causes : formulaire non rempli, erreur de saisie, donnée non disponible.  
Impact : la plupart des algorithmes ML ne gèrent pas les NaN nativement.
- [2] DOUBLONS  
Causes : import multiple, erreurs de fusion de bases.  
Impact : sur-représentation d'observations → biais d'apprentissage.
- [3] ERREURS DE SAISIE  
Exemples : "Brazzaville", "75 ans" pour un client de 7 ans,  
"1000000 FCFA" saisi comme "1 000 000" (espace = virgule ?).
- [4] INCOHÉRENCES  
Exemples : date de naissance postérieure à date d'embauche,  
revenus négatifs, client "féminin" avec nom masculin.
- [5] OUTLIERS (valeurs aberrantes)  
Peuvent être des erreurs ou de vraies observations extrêmes.  
Distinguer les deux est une compétence métier, pas seulement technique.
- [6] MAUVAIS TYPES  
Colonne "âge" stockée en texte (string) au lieu d'entier.  
Colonne "date" stockée comme texte au lieu de datetime.
- [7] COLONNES INUTILES  
ID techniques, colonnes entièrement vides, features sans variance.

---

### 5.3 PIPELINE DE NETTOYAGE EN PYTHON

---

```
import pandas as pd
import numpy as np

# --- CHARGEMENT ---
df = pd.read_csv("dataset_brut.csv")

# --- DIAGNOSTIC INITIAL ---
print(df.shape)          # (nb_lignes, nb_colonnes)
print(df.dtypes)         # Types de chaque colonne
print(df.isnull().sum()) # Nb de valeurs manquantes par colonne
print(df.duplicated().sum()) # Nb de doublons
print(df.describe())     # Statistiques descriptives

# --- SUPPRESSION DES DOUBLONS ---
```

```

df = df.drop_duplicates()

# --- CORRECTION DES TYPES ---
df["age"] = pd.to_numeric(df["age"], errors="coerce")
df["date_naissance"] = pd.to_datetime(df["date_naissance"],
                                     errors="coerce")

# --- TRAITEMENT DES VALEURS MANQUANTES ---

# Option 1 : Suppression (si peu de lignes concernées)
df = df.dropna(subset=["label"]) # Obligatoire : on ne peut pas imputer un label

# Option 2 : Imputation par la médiane (colonnes numériques)
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
df[["age", "revenu"]] = imputer.fit_transform(df[["age", "revenu"]])

# Option 3 : Imputation par le mode (colonnes catégorielles)
df["ville"].fillna(df["ville"].mode()[0], inplace=True)

# --- TRAITEMENT DES OUTLIERS ---
# Méthode IQR (Interquartile Range)
Q1 = df["revenu"].quantile(0.25)
Q3 = df["revenu"].quantile(0.75)
IQR = Q3 - Q1
df = df[~((df["revenu"] < Q1 - 1.5 * IQR) |
          (df["revenu"] > Q3 + 1.5 * IQR))]

# --- NORMALISATION / STANDARDISATION ---
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler() # Moyenne 0, écart-type 1
df[["age", "revenu"]] = scaler.fit_transform(df[["age", "revenu"]])

# --- ENCODAGE DES VARIABLES CATÉGORIELLES ---
df = pd.get_dummies(df, columns=["ville", "type_emploi"], drop_first=True)

# --- SAUVEGARDE ---
df.to_csv("dataset_propre.csv", index=False)
print(f"Dataset propre : {df.shape[0]} lignes, {df.shape[1]} colonnes")

```

---

## 5.4 FEATURE ENGINEERING : CRÉER DE LA VALEUR

---

Le feature engineering consiste à créer de nouvelles variables à partir des existantes pour améliorer la performance du modèle.

EXEMPLES CONCRETS :

Variables brutes : date\_naissance, date\_demande\_credit  
→ Feature créée : age\_au\_moment\_demande = date\_demande - date\_naissance

Variables brutes : revenu\_mensuel, montant\_credit  
→ Feature créée : ratio\_endettement = montant\_credit / revenu\_mensuel

Variables brutes : texte d'avis client  
→ Features créées : longueur\_texte, nb\_mots\_positifs, sentiment\_score

Variables brutes : latitude, longitude  
→ Feature créée : quartier\_ville (via clustering géographique)

Variables brutes : historique de transactions  
→ Features créées : nb\_transactions\_30j, montant\_moyen, écart-type des transactions, dernière activité

PRINCIPE : Un bon feature engineering vaut mieux qu'un algorithme plus complexe.

---

## PARTIE VI – ANNOTER ET ÉTIQUETER LES DONNÉES

---



---

### 6.1 POURQUOI L'ANNOTATION EST L'ENJEU MAJEUR DE L'IA SUPERVISÉE

---

Un label, c'est la réponse correcte.  
Sans labels corrects, l'IA n'apprend pas à être correcte.

L'annotation est coûteuse, lente et source d'erreurs humaines.  
C'est pourquoi les géants du tech (Google, Meta, Amazon) dépensent des centaines de millions de dollars pour annoter des données.

#### RÈGLE D'OR :

La qualité des labels est plus importante que la quantité de données.  
10 000 exemples correctement annotés > 100 000 exemples annotés approximativement.

---

## 6.2 TYPES D'ANNOTATIONS SELON LE DOMAINE

---

### TEXTE

- Classification de texte : attribuer une catégorie à un document  
Exemple : "positif / négatif / neutre" pour un avis client
- NER (Named Entity Recognition) : identifier les entités dans un texte  
Exemple : [PERSONNE]Ali Bongo[/PERSONNE] a rencontré [LIEU]Brazzaville[/LIEU]
- Question-Réponse : paires (question, texte source, réponse attendue)
- Résumé : paires (texte long, résumé)
- Instructions : paires (consigne, réponse idéale) → fine-tuning LLM

### IMAGE

- Classification : une image = une étiquette (chat / chien)
- Détection d'objet : bounding boxes (rectangles autour d'objets)
- Segmentation sémantique : chaque pixel a une étiquette
- Segmentation d'instance : distinctions entre instances du même objet
- Points clés (keypoints) : squelette humain, repères faciaux

### AUDIO

- Transcription : audio → texte
- Identification de langue
- Détection d'émotions vocales
- Classification de sons (bruit/musique/parole)

### VIDÉO

- Annotation temporelle : action qui commence à t=5s et finit à t=12s
- Tracking d'objets entre les frames
- Classification de scènes

---

## 6.3 OUTILS D'ANNOTATION

---

### TEXTE

- Label Studio (open-source, puissant, auto-hébergeable)
- Prodigy (payant, très efficace, intégré à spaCy)
- Doccano (open-source, simple)
- Amazon Mechanical Turk (crowdsourcing à grande échelle)

### IMAGE

- LabelImg (bounding boxes, gratuit)
- Roboflow (plateforme complète annotation + augmentation)
- CVAT (Computer Vision Annotation Tool, open-source)
- Supervisely (professionnel)

### AUDIO

- Audacity (simple, manuel)
- Label Studio (audio + transcription)
- Mozilla Common Voice (collecte + annotation communautaire)

---

## 6.4 BONNES PRATIQUES D'ANNOTATION

---

### [1] RÉDIGER UN GUIDE D'ANNOTATION (Annotation Guidelines)

Document qui définit précisément chaque label.

Exemple : "spam = tout message non sollicité à but commercial.

Inclut les promotions. Exclut les newsletters consenties."

### [2] VÉRIFIER L'ACCORD INTER-ANNOTATEURS

Faire annoter le même échantillon par plusieurs personnes.

Mesurer le taux d'accord (Cohen's Kappa, Fleiss' Kappa).  
Kappa > 0.8 = excellent accord. < 0.6 = guide insuffisant.

[3] COMMENCER PAR UN PILOTE

Annoter 100–200 exemples, mesurer l'accord, ajuster le guide,  
puis lancer la collecte à grande échelle.

[4] DOUBLE ANNOTATION + RÉOLUTION DES CONFLITS

Chaque exemple annoté par 2 personnes minimum.  
En cas de désaccord : troisième annotateur ou comité.

[5] TRACKING DE LA QUALITÉ

Quelques exemples "gold standard" (réponse connue) glissés  
incognito dans le batch pour mesurer la précision des annotateurs.

---

## PARTIE VII – ÉQUILIBRE, BIAIS ET REPRÉSENTATIVITÉ

---

---

### 7.1 LE PROBLÈME DES CLASSES DÉSÉQUILIBRÉES

---

Un dataset déséquilibré est un dataset où certaines classes  
sont beaucoup plus représentées que d'autres.

EXEMPLE TYPIQUE – Détection de fraude :

- 99% de transactions légitimes
- 1% de transactions frauduleuses

Un modèle naïf peut obtenir 99% de précision en disant  
TOUJOURS "légitime". Ce modèle est inutile.

SOLUTIONS :

[A] SOUS-ÉCHANTILLONNAGE (Undersampling)

Réduire la classe majoritaire.  
Risque : perdre des informations.

[B] SUR-ÉCHANTILLONNAGE (Oversampling)

Dupliquer ou synthétiser des exemples de la classe minoritaire.  
Technique SMOTE (voir Partie IV).

[C] PONDÉRATION DES CLASSES (Class Weights)

Pénaliser davantage les erreurs sur la classe minoritaire.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(class_weight="balanced")
```

[D] MÉTRIQUES ADAPTÉES

Ne pas utiliser l'accuracy sur des données déséquilibrées.  
Utiliser : F1-score, Précision/Rappel, AUC-ROC, Cohen's Kappa.

---

### 7.2 LES BIAIS DANS LES DATASETS

---

BIAIS DE SÉLECTION

Le dataset ne représente pas toute la population cible.  
Exemple : enquête en ligne dans un pays où peu de gens ont internet.  
→ Les répondants ne sont pas représentatifs.

BIAIS DE CONFIRMATION

On collecte des données qui confirment une hypothèse préexistante.  
→ La réalité complexe est simplifiée ou filtrée.

BIAIS HISTORIQUE

Les données passées reflètent des inégalités passées.  
Exemple : données de recrutement sur 20 ans dans une entreprise  
sexiste → l'IA reproduit la discrimination.

BIAIS DE MESURE

L'instrument de mesure introduit des erreurs systématiques.  
Exemple : capteur de glycémie moins précis sur peaux sombres.  
→ Données médicales biaisées selon la race.

## BIAIS DE LABEL

Les annotateurs introduisent leurs propres préjugés dans les étiquettes.  
Exemple : descriptions de visages annotées différemment selon la race.

## DÉTECTER ET RÉDUIRE LES BIAIS :

- Analyser les distributions par sous-groupe (sexe, âge, région, ethnie)
- Mesurer les métriques de performance par sous-groupe
- Documenter les biais connus dans le README
- Faire valider le dataset par des experts du domaine et des parties prenantes

---

## 7.3 REPRÉSENTATIVITÉ : LA RÉALITÉ DU TERRAIN DANS LE DATASET

---

Pour un dataset utilisé au Congo-Brazzaville, se poser les questions :

- Les données reflètent-elles les deux grandes villes (Brazza, Pointe-Noire) ET les zones rurales (Sangha, Lékoumou, Cuvette) ?
- Les langues locales sont-elles prises en compte (lingala, lari, etc.) ?
- Les réalités économiques locales (informal, mobile money) sont-elles représentées ?
- Les données saisonnières tiennent-elles compte du climat tropical ?
- Les disparités hommes/femmes dans l'accès aux services sont-elles visibles ?

Un dataset "internationalement standard" sera souvent biaisé pour le contexte congolais. Construire ses propres datasets est un avantage compétitif stratégique pour GCI.

---

## PARTIE VIII – FORMATS, STOCKAGE ET VERSIONNEMENT

---

---

### 8.1 LES FORMATS DE FICHIERS

---

| FORMAT   | EXTENSION   | CAS D'USAGE                     | AVANTAGES             |
|----------|-------------|---------------------------------|-----------------------|
| CSV      | .csv        | Données tabulaires universelles | Lisible partout       |
| JSON     | .json       | APIs, données hiérarchiques     | Flexible, universel   |
| Parquet  | .parquet    | Big data, colonnes              | Compression, rapide   |
| HDF5     | .h5         | Données volumineuses ML         | Structuré, binaire    |
| SQLite   | .db/.sqlite | Applications embarquées         | Léger, SQL complet    |
| JSONL    | .jsonl      | NLP, fine-tuning LLM            | Un exemple par ligne  |
| TFRecord | .tfrecord   | TensorFlow                      | Optimisé entraînement |
| Pickle   | .pkl        | Objets Python sérialisés        | Python-natif          |
| Arrow    | .arrow      | Échange de données rapide       | Zéro-copie            |

#### RECOMMANDATIONS :

- Partage et interopérabilité → CSV ou JSON
- Production et performance → Parquet
- Fine-tuning LLM → JSONL
- Entraînement ML classique → CSV ou Parquet
- Données volumineuses → Parquet + partitionnement

---

### 8.2 VERSIONNEMENT DES DATASETS

---

#### POURQUOI VERSIONNER ?

- Les datasets évoluent (nouvelles données, corrections)
- Il faut pouvoir reproduire exactement les résultats passés
- Tracer qui a modifié quoi et quand (auditabilité)

#### OUTILS :

DVC (Data Version Control)

Le "Git pour les données". S'intègre avec Git.

```
dvc init
dvc add data/dataset_clients.csv
git add data/dataset_clients.csv.dvc .gitignore
git commit -m "Ajout dataset clients v1.0"
```

```
# Envoyer les données vers un stockage distant
dvc remote add -d storage s3://mon-bucket/data
dvc push
```

## HUGGING FACE DATASETS

Pour publier et versionner des datasets accessibles publiquement.

## CONVENTION DE NOMMAGE MANUELLE (méthode simple)

```
dataset_clients_v1_0_2024_01.csv
dataset_clients_v1_1_2024_03_correction_doublons.csv
dataset_clients_v2_0_2024_06_nouvelles_features.csv
```

## 8.3 STRUCTURE DE RÉPERTOIRE RECOMMANDÉE

```
projet_ia/
├── data/
│   ├── raw/                ← Données brutes, jamais modifiées
│   │   └── clients_brut.csv
│   ├── interim/           ← Données en cours de traitement
│   │   └── clients_nettoye.csv
│   ├── processed/         ← Données finales prêtes pour le modèle
│   │   ├── train.csv
│   │   ├── val.csv
│   │   └── test.csv
│   └── external/          ← Données tierces (publiques, partenaires)
├── notebooks/             ← Exploration, EDA
├── src/
│   ├── data_collection.py
│   ├── data_cleaning.py
│   ├── feature_engineering.py
│   └── train.py
├── models/                ← Modèles entraînés sauvegardés
├── reports/               ← Visualisations, rapports
└── README.md
```

## RÈGLE FONDAMENTALE :

Le dossier raw/ est sacré. On ne le modifie JAMAIS.  
Toute transformation part du raw et produit un nouvel artefact.

## PARTIE IX – DATASETS POUR CHAQUE TYPE D'IA

### 9.1 DATASET POUR UN MODÈLE DE CLASSIFICATION

OBJECTIF : Prédire une catégorie (classe).

#### STRUCTURE MINIMALE :

- N colonnes de features
- 1 colonne label (catégorie)
- Au moins 1 000 exemples par classe
- Distribution des classes documentée

EXEMPLE – Détection de churn client :

| client_id | nb_achats_6m | anciennete_mois | nb_reclamations | churn |
|-----------|--------------|-----------------|-----------------|-------|
| C001      | 12           | 24              | 0               | 0     |
| C002      | 1            | 3               | 3               | 1     |
| C003      | 8            | 18              | 1               | 0     |

### 9.2 DATASET POUR UN MODÈLE DE RÉGRESSION

OBJECTIF : Prédire une valeur continue.

#### STRUCTURE :

- Features numériques et/ou catégorielles encodées
- 1 colonne cible numérique
- Vérifier la distribution de la cible (log-transformation si skewed)

EXEMPLE – Pr vision du chiffre d'affaires mensuel :

| mois_num | nb_employees | surface_m2 | zone_ville | ca_fcfa   |
|----------|--------------|------------|------------|-----------|
| 1        | 5            | 80         | 1          | 2 500 000 |
| 2        | 5            | 80         | 1          | 2 800 000 |
| 3        | 6            | 80         | 1          | 3 100 000 |

---

### 9.3 DATASET POUR UN MOD LE NLP

---

FORMATS SELON LA T CHE :

[A] Classification de texte (CSV)

| texte                                | label |
|--------------------------------------|-------|
| "Tr s bon service, livraison rapide" | 1     |
| "Produit ab m , d  u"                | 0     |

[B] Fine-tuning d'instruction (JSONL)

Format Alpaca (standard pour fine-tuning LLM) :

```
{"instruction": "R sum  ce texte",  
  "input": "Le Congo-Brazzaville est un pays...",  
  "output": "Le Congo est un pays d'Afrique centrale..."}
```

```
{"instruction": "Traduis en fran ais",  
  "input": "Hello, how are you?",  
  "output": "Bonjour, comment allez-vous ?"}
```

[C] Question-R ponse (JSON)

```
{  
  "context": "GCI est une soci t  de conseil bas e   Brazzaville...",  
  "question": "O  est bas e GCI ?",  
  "answer": "Brazzaville"  
}
```

---

### 9.4 DATASET POUR UN MOD LE DE VISION

---

STRUCTURE R PERTOIRE POUR CLASSIFICATION :

```
images/  
├── train/  
│   ├── manioc_sain/  
│   │   ├── img_001.jpg  
│   │   └── img_002.jpg  
│   └── manioc_malade/  
│       ├── img_001.jpg  
│       └── img_002.jpg  
└── val/  
    ├── manioc_sain/  
    └── manioc_malade/
```

FORMAT POUR D TECTION D'OBJET (YOLO) :

Chaque image a un fichier .txt associ  :

```
# classe x_centre y_centre largeur hauteur (normalis s 0-1)  
0 0.45 0.52 0.30 0.25  
1 0.78 0.33 0.15 0.20
```

AUGMENTATION DE DONN ES (pour l'image) :

Technique pour multiplier artificiellement un petit dataset.

```
from torchvision import transforms
```

```
augmentation = transforms.Compose([  
    transforms.RandomHorizontalFlip(p=0.5),  
    transforms.RandomRotation(degrees=15),  
    transforms.ColorJitter(brightness=0.2, contrast=0.2),  
    transforms.RandomCrop(224),  
    transforms.ToTensor()  
])
```

])

---

## 9.5 DATASET POUR SÉRIES TEMPORELLES

---

### STRUCTURE MINIMALE :

- Une colonne de dates (index temporel)
- Une ou plusieurs colonnes de valeurs
- Fréquence régulière (horaire, journalière, mensuelle)
- Pas de trous dans la série (ou gestion explicite des trous)

### EXEMPLE – Ventes quotidiennes :

| date       | ventes_fcfa | nb_clients | temperature |
|------------|-------------|------------|-------------|
| 2024-01-01 | 150 000     | 45         | 29.5        |
| 2024-01-02 | 130 000     | 38         | 30.1        |
| 2024-01-03 | 210 000     | 62         | 28.8        |

### CRÉATION EN PYTHON :

```
import pandas as pd
import numpy as np

# Générer une série synthétique
dates = pd.date_range(start="2023-01-01", end="2024-12-31", freq="D")
tendance = np.linspace(100000, 200000, len(dates))
saisonnalite = 20000 * np.sin(2 * np.pi * np.arange(len(dates)) / 365)
bruit = np.random.normal(0, 5000, len(dates))

df = pd.DataFrame({
    "date": dates,
    "ventes_fcfa": tendance + saisonnalite + bruit
})
df.to_csv("series_temporelle_ventes.csv", index=False)
```

---

## PARTIE X – ENJEUX ÉTHIQUES ET LÉGAUX

---

---

### 10.1 PROTECTION DES DONNÉES PERSONNELLES

---

DONNÉES PERSONNELLES = toute donnée permettant d'identifier une personne.  
Exemples : nom, email, téléphone, adresse, numéro de carte, localisation GPS.

### PRINCIPES FONDAMENTAUX :

- [1] CONSENTEMENT ÉCLAIRÉ  
La personne doit savoir et accepter que ses données soient collectées.  
Un formulaire d'enquête doit inclure une clause explicite.
- [2] MINIMISATION  
Ne collecter que les données strictement nécessaires à l'objectif.  
Si l'âge suffit, ne pas demander la date de naissance exacte.
- [3] ANONYMISATION  
Supprimer ou remplacer les identifiants directs avant de traiter.  
→ Supprimer nom, prénom, email, numéro de téléphone  
→ Remplacer l'ID par un hash non réversible
- [4] PSEUDONYMISATION  
Remplacer les identifiants par des codes ne permettant pas  
l'identification directe mais réversibles avec une clé.

### EXEMPLE PYTHON – ANONYMISATION :

```
import pandas as pd
import hashlib

df = pd.read_csv("clients_brut.csv")

# Pseudonymisation de l'ID
```



## 10.2 CADRE RÉGLEMENTAIRE

- RGPD (Europe) : référence mondiale sur la protection des données
- CCPA (Californie) : droits des consommateurs US
- PDPA (Asie du Sud-Est) : similaire au RGPD

- Convention de Malabo (Union Africaine) : framework continental
- Loi Informatique et Libertés du Congo-Brazzaville
- UEMOA / CEMAC : réglementations financières régionales

Même en l'absence d'application stricte locale, appliquer les principes RGPD constitue une bonne pratique professionnelle et un avantage concurrentiel auprès des clients exigeants.

Avant d'utiliser un dataset public, vérifier sa licence.

| LICENCE              | USAGE COMMERCIAL | MODIFICATION   | ATTRIBUTION REQUISE |
|----------------------|------------------|----------------|---------------------|
| CC0 (domaine public) | Oui              | Oui            | Non                 |
| CC BY                | Oui              | Oui            | Oui                 |
| CC BY-SA             | Oui              | Oui (même lic) | Oui                 |
| CC BY-NC             | NON              | Oui            | Oui                 |
| MIT                  | Oui              | Oui            | Oui                 |
| Apache 2.0           | Oui              | Oui            | Oui                 |
| Custom/Restrictive   | Vérifier         | Vérifier       | Vérifier            |

⚠ Utiliser un dataset CC BY-NC dans un projet commercial est une faute légale.

## PARTIE XI – PIPELINES DE DONNÉES EN PRODUCTION

## 11.1 DU NOTEBOOK AU PIPELINE RÉEL

Un notebook Jupyter est un outil d'exploration.

Un pipeline de production est un système automatisé, fiable et maintenu.

La transition de l'un à l'autre est le saut qualitatif entre un data scientist junior et un data scientist senior.

### PIPELINE BASIQUE :

```
graph LR
    A[COLLECTE] --> B[NETTOYAGE]
    B --> C[FEATURES]
    C --> D[ENTRAÎNEMENT]
    D --> E[ÉVALUATION]
    E --> F[DÉPLOIEMENT]
    F --> G[MONITORING ET FEEDBACK]
    G --> A
```

## 11.2 ORCHESTRATION AVEC APACHE AIRFLOW

Airflow permet de définir et programmer des pipelines de données (DAGs).

```
from airflow import DAG
from airflow.operators.python import PythonOperator
```

```

from datetime import datetime

def collecter_donnees():
    # Script de collecte
    pass

def nettoyer_donnees():
    # Script de nettoyage
    pass

def entrainer_modele():
    # Script d'entraînement
    pass

with DAG("pipeline_ia_gci",
        start_date=datetime(2024, 1, 1),
        schedule_interval="@weekly") as dag:

    t1 = PythonOperator(task_id="collecte", python_callable=collecter_donnees)
    t2 = PythonOperator(task_id="nettoyage", python_callable=nettoyer_donnees)
    t3 = PythonOperator(task_id="entraînement", python_callable=entrainer_modele)

    t1 >> t2 >> t3 # Ordre d'exécution

```

---

### 11.3 MONITORING : DÉTECTER LA DÉRIVE DES DONNÉES

---

**DATA DRIFT** : La distribution des données change par rapport à l'entraînement.

Exemple : inflation forte → les prix changent → le modèle de prix devient obsolète.

**CONCEPT DRIFT** : La relation entre features et label change.

Exemple : comportement d'achat post-COVID ≠ comportement pré-COVID.

**DÉTECTER LA DÉRIVE** :

```

from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=df_train, current_data=df_production)
report.save_html("drift_report.html")

```

**RÉPONSES À LA DÉRIVE** :

- Ré-entraîner le modèle sur des données récentes
- Mettre à jour le dataset avec de nouveaux exemples
- Réviser les features (certaines deviennent obsolètes)
- Alerter les équipes si la dérive dépasse un seuil

---

## PARTIE XII – DATASETS DANS LE CONTEXTE AFRICAIN ET CONGOLAIS

---



---

### 12.1 LE DÉSERT DE DONNÉES AFRICAIN : RÉALITÉ ET OPPORTUNITÉ

---

**RÉALITÉ** :

La grande majorité des datasets publics utilisés pour l'IA mondiale proviennent d'Europe, des États-Unis et d'Asie du Nord.  
Les données africaines sont massivement sous-représentées.

**CONSÉQUENCES** :

- Les IA déployées en Afrique performant moins bien sur les cas locaux
- Les langues africaines manquent de ressources NLP
- Les modèles médicaux ignorent les maladies tropicales dominantes
- Les modèles agricoles ignorent les cultures locales (manioc, plantain...)
- Les modèles financiers ignorent le secteur informel

**OPPORTUNITÉ POUR GCI** :

Construire des datasets congolais de référence est un avantage compétitif durable et un actif stratégique.  
La rareté de ces données les rend précieuses.

---

## 12.2 DATASETS PRIORITAIRES À CONSTRUIRE

---

### [COMMERCE ET MICROÉCONOMIE]

- Prix des marchés hebdomadaires (Brazzaville, Pointe-Noire)
- Flux de mobile money (MTN, Airtel Money)
- Catégories et volumes du commerce informel
- Historique de crédit des PME

### [AGRICULTURE]

- Rendements par culture, par zone, par saison
- Classification d'images de cultures (manioc, café, cacao)
- Données météo locales (ANAC, stations régionales)

### [SANTÉ]

- Données épidémiques anonymisées (paludisme, choléra, VIH)
- Images dermatologiques pour peaux sombres
- Données de consultation en centres de santé ruraux

### [LANGUE ET CULTURE]

- Corpus de textes en lingala, lari, kikongo, teke
- Transcriptions audio de radiodiffusion locale
- Paires de traduction lingala/lari ↔ français

### [URBANISME ET MOBILITÉ]

- Trafic routier Brazzaville (données GPS agrégées)
- Qualité de l'air par quartier
- Données cadastrales et plans d'urbanisme

---

## 12.3 STRATÉGIE DE COLLECTE LOCALE

---

### PARTENARIATS INSTITUTIONNELS

- INS (Institut National de la Statistique) : données officielles
- ANAC (météo) : séries climatiques
- Universités de Brazzaville et Pointe-Noire
- CHU, CHUB : données médicales anonymisées
- Mairies et communes : données administratives

### COLLECTE COMMUNAUTAIRE

- Applications mobiles légères (KoboToolbox, ODK)
- Réseau d'enquêteurs terrain formés par GCI
- Partenariats avec associations de commerçants

### DONNÉES NUMÉRIQUES DISPONIBLES

- Réseaux sociaux francophones (commentaires, posts)
- Presse en ligne congolaise (Les Dépêches de Brazzaville, Adiac)
- Forum et groupes WhatsApp (avec accord)
- Données gouvernementales open data (SNAT, plans nationaux)

---

## PARTIE XIII – CHECKLIST DU DATA SCIENTIST : DU DATASET AU MODÈLE

---

---

### 13.1 CHECKLIST COMPLÈTE

---

#### PHASE 1 – AVANT LA COLLECTE

- ☐ L'objectif métier est clairement défini par écrit
- ☐ Le type de tâche ML est identifié (classification, régression, etc.)
- ☐ La définition du label est précise et non ambiguë
- ☐ Les sources de données sont identifiées
- ☐ Les autorisations légales et éthiques sont obtenues
- ☐ Le schéma du dataset est documenté
- ☐ La taille cible du dataset est estimée

#### PHASE 2 – COLLECTE

- ☐ La collecte est automatisée et reproductible (scripts versionnés)
- ☐ Les données brutes sont sauvegardées (raw/, jamais modifié)
- ☐ Les métadonnées de collecte sont enregistrées (date, source, méthode)
- ☐ La conformité RGPD / locale est vérifiée

#### PHASE 3 – EXPLORATION (EDA)

- Distribution de chaque variable analysée (histogrammes, boxplots)
- Taux de valeurs manquantes par colonne calculé
- Distribution des classes (si supervisé) vérifiée
- Corrélations entre variables examinées
- Exemples individuels inspectés manuellement (eye-balling)
- Anomalies et outliers identifiés et documentés

#### PHASE 4 – NETTOYAGE ET PRÉPARATION

- Doublons supprimés
- Types de données corrigés
- Valeurs manquantes traitées (stratégie documentée)
- Outliers traités (strategy documentée)
- Variables catégorielles encodées
- Variables numériques normalisées si nécessaire
- Features inutiles supprimées

#### PHASE 5 – ANNOTATION (si supervisé)

- Guide d'annotation rédigé et validé
- Accord inter-annotateurs mesuré (Kappa > 0.7)
- Labels gold standard créés pour contrôle qualité
- Processus de résolution des conflits défini

#### PHASE 6 – SPLITS ET VALIDATION

- Train / Validation / Test splits créés (sans contamination)
- Distribution des classes vérifiée dans chaque split
- Données chronologiques : split temporel respecté (pas de data leakage)
- Splits sauvegardés séparément

#### PHASE 7 – DOCUMENTATION

- README rédigé (description, origine, limitations)
- Dictionnaire des données complété
- Biais connus documentés
- Licence et conditions d'utilisation précisées
- Version du dataset enregistrée (DVC ou convention de nommage)

#### PHASE 8 – VALIDATION FINALE

- Modèle baseline simple entraîné (logistic regression, random forest)
- Métriques cohérentes avec l'objectif (pas l'accuracy sur données déséquilibrées)
- Erreurs types analysées (confusion matrix, cas d'erreurs inspectés)
- Résultats documentés et reproductibles

---

### 13.2 SIGNAUX D'ALARME : QUAND LE DATASET EST PROBLÉMATIQUE

---

| SIGNAL                                      | PROBLÈME PROBABLE                            |
|---------------------------------------------|----------------------------------------------|
| Accuracy parfaite (99.9%) dès d'entrée      | Data leakage, colonnes futuristes            |
| Modèle ne surpasse pas le hasard            | Labels incorrects, features peu informatives |
| Performance excellente train, mauvaise test | Surapprentissage / trop peu de données       |
| Performance inégale selon les groupes       | Biais dans le dataset                        |
| Résultats non reproductibles                | Randomness non fixée (random_state)          |
| Les erreurs ne font pas sens métier         | Labels ambigus ou mal définis                |

---

### 13.3 RÈGLES D'OR DU DATA SCIENTIST PROFESSIONNEL

---

01. Ne jamais modifier les données brutes. JAMAIS.
02. Documenter chaque décision (pourquoi cette imputation ? ce seuil ?)
03. Le test set est sacré. Le regarder avant la décision finale invalide tout.
04. Méfier des datasets "trop propres" – la vraie vie est sale.
05. Un bon feature > un algorithme complexe.
06. La performance sur le benchmark ≠ la performance en production.
07. Comprendre ses données avant de coder. Toujours.
08. Visualiser avant de modéliser.
09. Le biais dans le dataset produit le biais dans l'IA. Responsabilité active.
10. Rendre son travail reproductible : seeds, versions, documentation.
11. Travailler avec les experts métier – les données sans contexte ne valent rien.
12. Construire le dataset minimum viable avant de viser l'exhaustivité.

---

### CONCLUSION – LA DONNÉE EST LE VRAI LEVIER

---

L'intelligence artificielle fascine par ses capacités apparemment magiques. Derrière chaque IA performante se cache un travail invisible mais décisif : la construction, le nettoyage, l'annotation et la gestion d'un dataset de qualité.

Dans le contexte africain, et congolais en particulier, ce travail prend une dimension supplémentaire : il s'agit de construire une représentation numérique fidèle de réalités qui n'ont jamais été capturées.

Celui qui maîtrise ses données maîtrise ses modèles.  
Celui qui maîtrise ses modèles maîtrise ses décisions.  
Celui qui maîtrise ses décisions construit l'avenir.

Pour GCI, la constitution de datasets de référence congolais est une mission stratégique autant qu'un travail technique.  
C'est la fondation sur laquelle tout le reste repose.

– GCI, Gykhamine Concept Investigation  
Congo-Brazzaville

---

## RESSOURCES POUR ALLER PLUS LOIN

---

### DATASETS PUBLICS DE RÉFÉRENCE

- Hugging Face Datasets Hub ([huggingface.co/datasets](https://huggingface.co/datasets))
- Kaggle Datasets ([kaggle.com/datasets](https://kaggle.com/datasets))
- UCI ML Repository ([archive.ics.uci.edu](https://archive.ics.uci.edu))
- Google Dataset Search ([datasetsearch.research.google.com](https://datasetsearch.research.google.com))
- Zenodo ([zenodo.org](https://zenodo.org)) : publications scientifiques avec données
- African Open Data ([africaopendata.org](https://africaopendata.org))
- Data for Africa ([dataforafrica.org](https://dataforafrica.org))

### OUTILS ESSENTIELS

- Pandas : manipulation données tabulaires
- DVC : versionnement de datasets
- Label Studio : annotation multi-tâches
- Great Expectations : validation qualité des données
- Evidently AI : monitoring de la dérive
- Roboflow : annotation et augmentation images
- Faker / SDV : génération données synthétiques
- Apache Airflow : orchestration de pipelines

### LECTURES RECOMMANDÉES

- "The Data Engineering Cookbook" – Andreas Kretz
- "Designing Machine Learning Systems" – Chip Huyen
- "Data Feminism" – D'Ignazio & Klein (biais et représentation)
- "Building Machine Learning Powered Applications" – Emmanuel Ameisen
- "Practical Data Science with Python" – Yuli Vasiliev

### CONFÉRENCES ET PUBLICATIONS

- NeurIPS Datasets and Benchmarks Track
- ICLR : recherches sur l'apprentissage représentationnel
- AI for Good (ITU) : IA et développement durable
- Deep Learning Indaba : communauté ML africaine
- Masakhane : NLP pour les langues africaines ([masakhane.io](https://masakhane.io))

---

FIN DU COURS – GCI © 2024-2025 – TOUS DROITS RÉSERVÉS  
Formation continue disponible via GCI – Congo-Brazzaville

---